PRODUCTION DOCUMENTATION

Comprehensive technical reference covering Postmark email integration, Stripe billing, authentication, API standards, environment configuration, and production deployment procedures.

VERSION 1.1 | APRIL 2026

Table of Contents

1. Postmark Email Integration Reference

The Renewably CRM uses Postmark as its transactional email service for sending proposals, invoices, contact notifications, and welcome emails. The integration is implemented through a custom HTTP client library located at `src/lib/postmark.ts` that communicates directly with the Postmark REST API. This section provides the complete reference for understanding, configuring, and extending the email integration in production. All email functions support graceful degradation: if the Postmark server token is not configured, the system logs a warning and returns a synthetic response instead of crashing.

1.1 Architecture Overview

The email system follows a modular architecture where the core Postmark client handles HTTP communication, and specialized wrapper functions provide domain-specific email sending capabilities. There is no Postmark SDK dependency in the project; the integration uses native fetch for all API calls. The API route at `/api/crm/email` handles email activity logging to the database, while the actual sending is delegated to specialized functions triggered from other routes (proposals, invoices, contact forms). Open tracking is enabled by default on all outbound emails, and link tracking is set to HTML-only mode to preserve plain text link integrity.

Component	Location	Purpose
Postmark Client	src/lib/postmark.ts	Core HTTP client, sendEmail(), sendTemplate()
Email API Route	src/app/api/crm/email/route.ts	POST: log email activity to DB
Contact Notification	src/lib/postmark.ts	sendContactNotification() - form submissions
Welcome Email	src/lib/postmark.ts	sendWelcomeEmail() - auto-reply to contacts
Proposal Email	src/lib/postmark.ts	sendProposalEmail() - proposal delivery
Invoice Email	src/lib/postmark.ts	sendInvoiceEmail() - invoice delivery

1.2 Core Postmark Client Functions

The core client provides two primary functions for sending emails. The `sendEmail()` function sends raw HTML emails and is the foundation for all wrapper functions. The `sendTemplate()` function sends emails using Postmark server-side templates, which is the recommended approach for consistent branding. Both functions accept a standard options object and return a response with message ID, submission timestamp, and error details.

```
// sendEmail(options) - Raw HTML email<br/>// Endpoint: POST
https://api.postmarkapp.com/email/withTemplate<br/>interface SendEmailOptions
{<br/>  From: string;        &nbs
p;         // Defaults to FROM_EMAIL
env var<br/>  To: string | string[];<br/>  Cc?: string |
string[];<br/>  Bcc?: string | string[];<br/>  ReplyTo?:
string;<br/>  Subject: string;<br/>  HtmlBody:
string;<br/>  TextBody?: string;<br/>  Tag?: string;  
;             //
For analytics categorization<br/>  Attachments?:
Attachment[];<br/>  TrackOpens?: boolean;    //
Default: true<br/>  TrackLinks?:
```

```
string;       // Default: "HtmlOnly"<br/>}
```

```
// sendTemplate(options) - Postmark server-side template<br/>// Endpoint: POST
https://api.postmarkapp.com/email/send<br/>interface SendTemplateOptions
{<br/>  From: string;<br/>  To: string |
string[];<br/>  TemplateId: number;   // Postmark template
ID<br/>  TemplateModel: Record;&lt;string,
unknown&gt;;<br/>  Tag?: string;<br/>  InlineCss?:
boolean;    // Default: true<br/>}
```

1.3 Email Tag Conventions

Tags are used to categorize emails in the Postmark dashboard for analytics and debugging. The CRM uses a consistent naming convention with lowercase letters and hyphens. Each business action that triggers an email has its own unique tag. When creating new email functions, always include a descriptive tag following this pattern. Tags appear in the Postmark activity feed, bounce reports, and can be used to filter open/click statistics by email type.

Tag	Trigger	Used By	Recipients
contact-form	Contact form submission	sendContactNotification()	hello@renewably.ie
welcome-auto-reply	New contact created	sendWelcomeEmail()	Contact email
proposal-sent	Proposal sent to client	sendProposalEmail()	Client email
invoice-sent	Invoice issued to client	sendInvoiceEmail()	Client email

1.4 Email Activity Logging

The POST `/api/crm/email` route does not send emails through Postmark directly. Instead, it logs email activities to the database as Activity records with type "email". This is called from various places in the CRM to maintain a complete communication history for each contact. The route validates the recipient email using a regex pattern, ensures subject and body are present, and optionally links the activity to a contact via contactId. This route is rate-limited to 10 requests per minute per IP address.

```
// POST /api/crm/email - Log email activity<br/>Request
Body:<br/>{<br/>  "to": "client@example.com",    //
Required, validated by email regex<br/>  "subject": "Proposal for Solar
Installation",<br/>  "body": "&lt;p&gt;Dear
John...&lt;/p&gt;",  // Required, HTML
supported<br/>  "contactId": "clx_abc123",    //
Optional<br/>  "companyId": "clx_def456",    //
Optional<br/>  "dealId":
"clx_ghi789"        //
Optional<br/>}<br/>Response (200): { "id": "act_xyz", "type": "email", "createdAt":
"..." }
```

1.5 Postmark Webhook Setup

For production, configure Postmark webhooks to receive delivery and bounce notifications. These should be pointed to a dedicated webhook endpoint on your server. While the current schema includes an email_log table with Postmark-specific fields (message_id, postmark_tag, delivery_status), the webhook handler implementation is a recommended addition for the production deployment. The webhook should process delivery events, bounces, spam complaints, and

opens/clicks to maintain accurate email delivery records in the CRM.

Webhook Event	Recommended Action	DB Update
Delivery	Mark email as delivered	email_log.status = "delivered"
Bounce (hard)	Flag contact email as invalid	contact.emailInvalid = true
Bounce (soft)	Retry after cooldown	email_log.status = "soft_bounce"
Spam Complaint	Alert admin, flag contact	contact.spamReport = true
Open	Track engagement	email_log.openedAt = timestamp
Click	Track link engagement	email_log.clickedAt = timestamp

2. Environment Variables and Configuration

This section documents every environment variable required or recommended for running the Renewably CRM in production. The application currently uses a minimal .env file with only the database URL for local SQLite development. When migrating to Supabase PostgreSQL, several additional variables must be configured. Variables are grouped by service: database, authentication, email, payments, calendar, and application settings.

2.1 Required Environment Variables

Variable	Required	Default	Description
DATABASE_URL	Yes	-	Prisma connection string (PostgreSQL for production)
NODE_ENV	Yes	development	Set to "production" for production builds
STRIPE_SECRET_KEY	Yes	-	Stripe API secret key for payment processing
STRIPE_WEBHOOK_SECRET	Yes	-	Stripe webhook signing secret
POSTMARK_SERVER_TOKEN	Yes	-	Postmark API server token for email
FROM_EMAIL	Yes	hello@renewably.ie	Default sender email address
NEXT_PUBLIC_APP_URL	Yes	-	Public app URL (e.g. https://crm.renewably.ie)

2.2 Stripe Plan Price IDs

The CRM uses three subscription tiers (Starter, Pro, Enterprise) with corresponding Stripe price IDs. These must be created in the Stripe dashboard after setting up products and pricing plans. The price IDs are used by the `getPriceIdForPlan()` function in `src/lib/stripe.ts`.

Variable	Plan	Description
STRIPE_PRICE_STARTER	Starter	Monthly/annual price ID for starter tier
STRIPE_PRICE_PRO	Pro	Monthly/annual price ID for pro tier
STRIPE_PRICE_ENTERPRISE	Enterprise	Monthly/annual price ID for enterprise tier

2.3 Optional Environment Variables

Variable	Default	Description
REDIS_URL	localhost:6379	Redis connection for caching/sessions
GOOGLE_CLIENT_ID	-	Google OAuth2 for calendar integration
GOOGLE_CLIENT_SECRET	-	Google OAuth2 client secret
LOG_LEVEL	info	Minimum log level: debug/info/warn/error
ANTHROPIC_API_KEY	-	Claude AI for analytics health checks

2.4 Production .env.example

```
# Database (Supabase PostgreSQL)<br/>DATABASE_URL="postgresql://postgres:[PASSWORD]
@db.[PROJECT].supabase.co:5432/postgres"<br/># Application<br/>NODE_ENV="production
"<br/>NEXT_PUBLIC_APP_URL="https://crm.renewably.ie"<br/>LOG_LEVEL="info"<br/>#
Email (Postmark)<br/>POSTMARK_SERVER_TOKEN="[YOUR_POSTMARK_SERVER_TOKEN]"<br/>FROM_
EMAIL="hello@renewably.ie"<br/># Payments (Stripe)<br/>STRIPE_SECRET_KEY="sk_live_[
...]"<br/>STRIPE_WEBHOOK_SECRET="whsec_[...]"<br/>STRIPE_PRICE_STARTER="price_[...]
"<br/>STRIPE_PRICE_PRO="price_[...]"<br/>STRIPE_PRICE_ENTERPRISE="price_[...]"<br/>
# Google Calendar (optional)<br/>GOOGLE_CLIENT_ID="[CLIENT_ID]"<br/>GOOGLE_CLIENT_S
ECRET="[CLIENT_SECRET]"<br/># Redis
(optional)<br/>REDIS_URL="redis://localhost:6379"
```

3. Authentication and Session Security

The CRM implements a custom token-based session authentication system using bcryptjs for password hashing and crypto for secure session token generation. This is not a JWT-based system; sessions are stored in the database and validated on every request. The authentication architecture provides multiple layers of security including rate limiting at both middleware and route levels, HttpOnly cookies to prevent XSS-based token theft, and automatic session expiration after 7 days.

3.1 Authentication Flow

The login process follows a five-step sequence. First, the middleware rate limiter checks the IP address against a sliding window of 10 requests per minute for the login endpoint specifically. Next, the request body is parsed to extract the email (lowercased) and password. The system then performs a database lookup to find the user by email address. If found, the password is verified against the stored hash using bcrypt.compare() with the original 12-round salt. Upon successful verification, a 32-byte random hex token is generated via crypto.randomBytes(32) and stored as a Session record with a 7-day expiry. The token is set as an HttpOnly cookie with SameSite=Lax and the Secure flag in production.

Step	Action	Implementation Detail
1	Rate limit check	In-memory IP limiter: 5 attempts / 60 seconds
2	Parse credentials	Email lowercased, password extracted from body
3	Database lookup	Prisma: db.user.findUnique({ where: { email } })
4	Password verify	bcrypt.compare(password, user.passwordHash) - 12 rounds
5	Session create	crypto.randomBytes(32) -> hex token, 7-day TTL
6	Set cookie	crm_session=; HttpOnly; SameSite=Lax; Secure (prod)

3.2 Session Configuration

Parameter	Value	Notes
Cookie Name	crm_session	Used by requireAuth() to validate requests
Token Length	64 hex characters	32 bytes encoded as hex via crypto.randomBytes(32)
Session TTL	7 days (604800 seconds)	Max-Age cookie attribute
Cookie Flags	HttpOnly, SameSite=Lax	Secure flag added in production
Storage	Prisma Session table	Fields: userId, token, expiresAt
Hash Rounds	12	bcrypt salt rounds for password hashing

3.3 Auth Guards in API Routes

Two authentication guard functions are available for protecting API routes. The `requireAuth(request)` function reads the session cookie, validates it against the database, checks expiration, and returns the user object. The `requireAdmin(request)` function performs the same validation but additionally checks that the user role is "admin", returning a 403 Forbidden response for non-admin users. Both functions are imported from `src/lib/crm-auth.ts`.

```
import { requireAuth, requireAdmin, unauthorized } from
"@/lib/crm-auth";<br/><br/>// Standard auth guard<br/>export async function
GET(request: Request) {<br/>  const user = await
requireAuth(request);<br/>  if (!user) return
unauthorized();<br/>  // ... proceed with authenticated
logic<br/>}<br/><br/>// Admin-only guard<br/>export async function PATCH(request:
Request) {<br/>  const user = await
requireAdmin(request);<br/>  if (!user) return NextResponse.json({ error:
"Admin required" }, { status: 403 });<br/>  // ... proceed with admin
logic<br/>}
```

3.4 Rate Limiting Strategy

The CRM implements dual-layer rate limiting. The first layer is at the Next.js middleware level (`src/middleware.ts`), which applies to all routes matching `/crm/*` and `/api/crm/*`. The second layer is at the individual route level, where each route handler implements its own rate limiting with configurable thresholds.

Route / Endpoint	Rate Limit	Window
Login (middleware)	10 requests	Per minute per IP
Login (route-level)	5 attempts	60 seconds per IP
Email logging	10 requests	Per minute per IP
Billing checkout	5 requests	5 minutes per IP
Call logging	30 requests	Per minute per IP

4. Stripe Billing Integration

The CRM integrates with Stripe for subscription billing management across three plan tiers. The billing system is split across five API sub-routes rather than a single monolithic route. The Stripe client is initialized as a singleton with API version 2025-04-30.basil and provides functions for customer management, checkout sessions, subscription lifecycle, portal management, and webhook verification. When a new installer profile is created, a 14-day trial subscription is automatically provisioned.

4.1 Billing API Routes

Method	Route	Description	Auth
GET	/api/crm/billing/plans	Returns Starter/Pro/Enterprise plan definitions	Required
POST	/api/crm/billing/checkout	Creates Stripe Checkout Session	Required
POST	/api/crm/billing/portal	Creates Stripe Customer Portal session	Required
GET	/api/crm/billing/status	Returns subscription status for installer	Required
POST	/api/crm/billing/webhook	Processes Stripe webhook events	Signature verified

4.2 Subscription Lifecycle

The subscription lifecycle begins when an installer profile is created, which automatically provisions a 14-day trial subscription. The installer can then upgrade to a paid plan through the Stripe Checkout flow. Stripe manages the recurring billing, and webhook events keep the CRM database synchronized. When a customer cancels, the subscription continues until the end of the current billing period (cancel_at_period_end). The billing portal allows installers to self-manage payment methods, view invoices, and cancel subscriptions without admin intervention.

4.3 Stripe Webhook Events

The webhook endpoint at POST `/api/crm/billing/webhook` is unauthenticated but verified using the Stripe webhook signing secret. The raw request body is passed to `stripe.webhooks.constructEvent()` with the signature header to validate authenticity.

Stripe Event	Action Taken	Database Update
checkout.session.completed	Create/upsert Subscription, save Stripe customer ID	subscriptions + installers tables
customer.subscription.updated	Sync status, period dates, cancellation flag	subscriptions table
customer.subscription.deleted	Mark subscription as canceled	subscriptions.status = "canceled"
invoice.payment_failed	Set subscription to past_due	subscriptions.status = "past_due"
invoice.payment_succeeded	Reactivate, sync period dates	subscriptions.status = "active"

4.4 Checkout Session Creation

```
// POST /api/crm/billing/checkout<br/>Request
Body:<br/>{<br/>  "installerId":
"clx_installer123",<br/>  "planId": "pro"  // "starter" | "pro"
| "enterprise"<br/>}<br/>Response (200):<br/>{<br/>  "checkoutUrl":
"https://checkout.stripe.com/c/pay/cs_live_...",<br/>  "sessionId":
"cs_live_..."<br/>}
```

5. API Standards and Error Handling

All CRM API routes follow a consistent pattern for request validation, response formatting, error handling, and logging. Every route wraps its handler in a try/catch block that logs errors via the structured JSON logger and returns a generic 500 response to clients, preventing internal details from leaking.

5.1 Standard Error Response Format

HTTP Status	Error Type	Response Shape
400	Validation Error	{ error: "Validation failed", details: { fieldErrors: {}, formErrors: [] } }
401	Unauthorized	{ error: "Unauthorized" }
403	Forbidden	{ error: "Admin access required" }
404	Not Found	{ error: "Deal not found" }
409	Conflict	{ error: "Installer profile already exists" }
429	Rate Limited	{ error: "Too many requests" } + Retry-After header
500	Server Error	{ error: "Internal server error" }

5.2 Route Handler Pattern

Every API route handler follows the same structural pattern. The handler begins with optional authentication via requireAuth() or requireAdmin(). Next, the request body is parsed and validated using the appropriate Zod schema from `src/lib/crm-schemas.ts`. If validation fails, a 400 response is returned with the flattened error details. The main business logic is wrapped in a

try/catch block.

5.3 Pagination Convention

All list endpoints (GET routes that return collections) support a standardized pagination format. The default page size is 50 with a maximum of 100. All list endpoints return a consistent response envelope with data, pagination metadata, and optional stats.

Parameter	Type	Default	Constraints
page	integer	1	Minimum: 1
limit	integer	50	Minimum: 1, Maximum: 100
search	string	-	Free-text search across name/email fields
sort	string	createdAt	Field name for sorting
order	string	desc	"asc" or "desc"

6. Complete API Route Reference

This section provides a comprehensive reference table of all CRM API routes, their supported HTTP methods, authentication requirements, and a brief description of each endpoint. All routes are prefixed with `/api/crm/` and require authentication unless otherwise noted.

Route	Methods	Auth	Description
/auth	GET, POST, DELETE	No (POST/DEL)	Login, session check, logout
/activities	GET, POST	Yes	List/create deal activities
/analytics/website	GET	Yes	Dashboard analytics + health checks
/ai	POST	Yes	AI assistant chat (z-ai-web-dev-sdk)
/call	POST	Yes	Log phone call activity
/calendar/google/*	Various	Yes	Google Calendar OAuth + sync
/companies	GET, POST	Yes	List/create companies
/contacts	GET, POST	Yes	List/create contacts
/dashboard	GET	Yes	Comprehensive dashboard KPIs
/deals	GET, POST	Yes	List/create deals
/financial	GET	Yes	Financial analytics (ARR/MRR)
/installers	GET, POST	Yes	List/create installer profiles
/invoices	GET, POST	Yes	List/create invoices
/leads	GET, POST	Yes	List/create leads
/meetings	GET, POST	Yes	List/create meetings

Route	Methods	Auth	Description
/notes	GET, POST	Yes	List/create notes
/pipeline	GET, PUT	Yes	Kanban pipeline view + stage update
/proposals	GET, POST	Yes	List/create proposals
/reports	GET, POST	Yes	List/create saved reports
/settings	PATCH	Admin	Update admin profile
/stats	GET	Yes	Lead-focused dashboard stats
/tags	GET, POST, DELETE	Yes	List/create/delete tags
/tasks	GET, POST, PUT	Yes	List/create/update tasks
/workflows	GET, POST	Yes	List/create workflow rules
/billing/*	Various	Mixed	Plans, checkout, portal, webhooks

7. Data Model Quick Reference

The CRM data model is defined by Zod schemas in `src/lib/crm-schemas.ts` and backed by Prisma with SQLite for development and PostgreSQL (Supabase) for production. The Supabase SQL schema (delivered separately as supabase-schema.sql) translates these Zod definitions into PostgreSQL tables with proper types, constraints, indexes, and Row-Level Security policies.

7.1 Key Entity Types and Fields

Entity	Key Fields	Validation Rules
Company	name*, counties, teamSize, status	Status: prospect/active/inactive/churned
Contact	firstName*, lastName*, email*, phone	Email: max 300 chars, lowercased
Deal	companyId*, product, stage*, mrr	9 stages, currency max 99999999
Lead	firstName*, lastName*, email*, source	Auto-assigned to current user, status=new
Task	title*, priority, dueDate, assignee	Priority: low/medium/high
Invoice	contactId*, lineItems[], taxRate	Auto-numbered: INV-YYYY-###
Proposal	title*, dealId, lineItems[]	Auto-calculated total
Meeting	title*, date*, meetingType	5 types, optional follow-up task
Installer	~50 fields	Business, billing, SEAI, certifications, fleet
Workflow	triggerType, actions[]	13 triggers, 9 action types

7.2 Enum Types Reference

The following enum values are used throughout the application. In the Prisma schema (development), these are represented as string literals. In the Supabase PostgreSQL schema (production), these are defined as PostgreSQL enum types for data integrity and query performance.

Enum Category	Values
Deal Stage	new_lead, contacted, discovery_call, demo_booked, demo_done, proposal_sent, negotiation, closed_won, closed_lost
Company Status	prospect, active, inactive, churned
Task Priority	low, medium, high
Task Status	pending, in_progress, completed
Lead Status	new, contacted, qualified, proposal, negotiation, won, lost
Meeting Type	call, video, in_person, demo, other
Meeting Status	scheduled, completed, cancelled, no_show
Invoice Status	draft, sent, paid, overdue, cancelled
Subscription Status	active, trialing, past_due, canceled, unpaid
Workflow Trigger	deal_stage_change, deal_created, new_contact, contact_inactive, task_overdue, task_completed, proposal_status_change, meeting_created, meeting_completed, meeting_cancelled, invoice_created, invoice_overdue, payment_received
Workflow Action	create_task, send_email, update_field, add_note, notify, create_meeting, create_proposal, create_invoice, create_note

8. Production Deployment Checklist

This section provides a comprehensive checklist for deploying the Renewably CRM to a self-hosted production server. The application uses Next.js 16 with the App Router, Prisma ORM, and is configured for standalone output mode. The build process creates an optimized standalone server bundle that runs directly on a VPS or dedicated server with Bun or Node.js. This guide covers the complete deployment pipeline from pre-flight checks through process management and monitoring.

8.1 Pre-Deployment Checklist

Before deploying, ensure all the following prerequisites are met. Each item should be verified and signed off by the development lead or DevOps engineer responsible for the deployment.

Step	Task	Details
1	Set up Supabase project	Create project at supabase.com, note Project URL and anon key
2	Run SQL schema	Execute supabase-schema.sql in Supabase SQL Editor
3	Enable RLS policies	Uncomment and enable Row-Level Security policies
4	Create admin user	Run seed data or create admin via auth endpoint

Step	Task	Details
5	Configure Postmark	Create server token, set up sender domain DNS records
6	Set up Stripe	Create products/prices, configure webhook endpoint
7	Provision server	VPS with 2+ vCPU, 4GB+ RAM, Ubuntu 22.04+ recommended
8	Build and test	Run build locally, verify 0 errors
9	Configure domain	Point A record to server IP
10	Set up monitoring	Configure uptime monitoring and error alerting

8.2 Database Migration

The application currently uses SQLite for development. Migrating to Supabase PostgreSQL requires executing the provided `supabase-schema.sql` file in the Supabase SQL Editor. This creates all tables, enums, indexes, triggers, and seed data. Data migration from SQLite to PostgreSQL is not automated; if there is existing development data to preserve, use a tool like pgloader or export/import scripts.

```
# Step 1: Execute SQL schema in Supabase SQL Editor<br/># (Copy contents of
supabase-schema.sql)<br/><br/># Step 2: Update DATABASE_URL in .env<br/>DATABASE_UR
L="postgresql://postgres:[PASSWORD]@db.[PROJECT].supabase.co:5432/postgres"<br/><br
/># Step 3: Update Prisma schema provider<br/># Change: provider = "sqlite"<br/>#
To: provider = "postgresql"<br/><br/># Step 4: Regenerate Prisma client<br/>npx
prisma generate<br/><br/># Step 5: Verify connection<br/>npx prisma db pull
```

8.3 Build Configuration

The Next.js application is configured for standalone output mode in `next.config.ts`, which produces a self-contained server bundle that includes all necessary dependencies. The build script in `package.json` runs `next build` followed by copying static assets and the public directory into the standalone output folder. The production start command uses Bun to run the standalone server at `.next/standalone/server.js`.

Command	Purpose	Notes
npm run build	Build + copy static assets to standalone	Output: .next/standalone/
npm run start	Start production server with Bun	Runs on port 3000
npm run dev	Development server with hot reload	Port 3000, not for production
npx prisma generate	Regenerate Prisma client	Required after schema changes
npx prisma db push	Push schema changes to database	Use carefully in production

8.4 Self-Hosted Server Deployment

The Renewably CRM is designed to run on a self-hosted server using the Next.js standalone output mode. This approach gives full control over the runtime environment, avoids vendor lock-in, and is cost-effective for a single-tenant application. The standalone build produces a self-contained server bundle in `.next/standalone/` that includes all Node.js dependencies, requiring only Bun or Node.js on the target machine. The recommended stack is Ubuntu 22.04 LTS with Caddy as the reverse

proxy (providing automatic HTTPS via Let's Encrypt), systemd or PM2 for process management, and optional Redis for session caching and rate limiting.

Server Requirements

Resource	Minimum	Recommended
CPU	1 vCPU	2+ vCPU
RAM	2 GB	4 GB
Disk	20 GB SSD	40 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Runtime	Node.js 18+ or Bun	Bun (latest)
Reverse Proxy	Caddy 2.x	Caddy 2.x

Deployment Steps

Step 1: Install runtime and dependencies. Connect to your server via SSH and install Bun (or Node.js) along with essential build tools. Bun is recommended for its significantly faster startup times and lower memory footprint compared to Node.js.

```
# Install Bun (recommended)<br/>curl -fsSL https://bun.sh/install | bash<br/># Or install Node.js 22 LTS<br/>curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -<br/>sudo apt-get install -y nodejs
```

Step 2: Clone the repository and install dependencies. Clone the project to a suitable directory such as `/opt/renewably`, then install all production dependencies using Bun.

```
sudo mkdir -p /opt/renewably<br/>sudo chown $USER:$USER /opt/renewably<br/>cd /opt/renewably<br/>git clone [REPO_URL] .<br/>bun install --production
```

Step 3: Configure environment variables. Create a `.env` file in the project root with all production values. Ensure DATABASE_URL points to your Supabase PostgreSQL instance. The .env file should have restricted permissions (600).

```
cp .env.example .env<br/>nano .env<br/>chmod 600 .env
```

Step 4: Build the application. Run the production build command which compiles the Next.js application with standalone output, optimises assets, and copies static files into the standalone bundle directory. Verify the build completes with zero errors.

```
bun run build
```

Step 5: Test the standalone server. Before setting up the reverse proxy and process manager, verify that the standalone server starts correctly and responds on port 3000.

```
NODE_ENV=production bun .next/standalone/server.js<br/># In another terminal:<br/>curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/<br/># Expected: 200
```

start.sh Startup Script

The project includes a `start.sh` script that automates the server startup process. This script sets the NODE_ENV environment variable, changes to the project directory, and starts the standalone server using Bun.

```
#!/bin/bash<br/># start.sh - Production startup script<br/>export NODE_ENV=production<br/>cd /opt/renewably<br/>exec bun .next/standalone/server.js
```

keep-alive.sh Monitoring Script

The `keep-alive.sh` script provides basic process monitoring by checking if the server is responding with HTTP 200. If the server becomes unresponsive, it will restart the process automatically. This script is suitable for cron-based monitoring or as a supplement to systemd or PM2.

```
#!/bin/bash<br/># keep-alive.sh - Process monitoring and
auto-restart<br/>STATUS=$(curl -s -o /dev/null -w "%{http_code}"
http://localhost:3000/)<br/>if [ "$STATUS" != "200" ]; then<br/>  echo
"Server down (HTTP $STATUS), restarting..."<br/>  cd
/opt/renewably<br/>  pkill -f
"standalone/server.js"<br/>  NODE_ENV=production nohup bun
.next/standalone/server.js > /var/log/renewably.log 2>&1 &<br/>fi
```

8.5 Caddy Reverse Proxy Setup

Caddy is the recommended reverse proxy for the Renewably CRM. It provides automatic HTTPS certificate provisioning and renewal via Let's Encrypt, requires minimal configuration, and handles gzip compression and static file serving out of the box. The project includes a `Caddyfile` in the repository root that can be adapted for production use.

Caddy Installation

```
# Install Caddy (Ubuntu/Debian)<br/>sudo apt install -y debian-keyring
debian-archive-keyring apt-transport-https<br/>curl -1sLf
"https://dl.cloudsmith.io/public/caddy/stable/gpg.key" | sudo gpg --dearmor -o
/usr/share/keyrings/caddy-stable-archive-keyring.gpg<br/>curl -1sLf
"https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt" | sudo tee
/etc/apt/sources.list.d/caddy-stable.list<br/>sudo apt update<br/>sudo apt install
caddy
```

Caddyfile Configuration

The following Caddyfile configuration routes traffic to the Next.js standalone server on port 3000, enables automatic HTTPS for the specified domain, sets security headers, and handles WebSocket proxying required for real-time features. Replace `crm.renewably.ie` with your actual production domain.

```
crm.renewably.ie {<br/>  reverse_proxy
localhost:3000<br/><br/>  # Security headers<br/>  header
X-Frame-Options "SAMEORIGIN"<br/>  header X-Content-Type-Options
"nosniff"<br/>  header Referrer-Policy
"strict-origin-when-cross-origin"<br/>  header X-XSS-Protection "1;
mode=block"<br/>  header Permissions-Policy "camera=(), microphone=(),
geolocation=()"<br/><br/>  # Static asset caching<br/>  @static
path /_next/static/*<br/>  header @static Cache-Control "public,
max-age=31536000, immutable"<br/><br/>  # Image optimization
cache<br/>  @images path /_next/image/*<br/>  header @images
Cache-Control "public, max-age=86400,
stale-while-revalidate=31536000"<br/><br/>  #
Compression<br/>  encode gzip zstd<br/>}
```

Enabling the site. After placing the Caddyfile at `/etc/caddy/Caddyfile`, validate the configuration and reload Caddy to apply changes. Caddy will automatically obtain and renew the TLS certificate from Let's Encrypt.

```
sudo caddy validate --config /etc/caddy/Caddyfile<br/>sudo systemctl reload
caddy<br/>sudo systemctl status caddy
```

8.6 Process Management and Monitoring

For production reliability, the Next.js server should be managed by a process supervisor that automatically restarts it on crashes, manages logs, and ensures the service starts on boot. Two options are recommended: systemd (built into Ubuntu, zero additional dependencies) or PM2 (Node.js process manager with additional features like clustering and monitoring dashboard).

Option A: systemd Service

Create a systemd service unit file for the Renewably CRM. This is the simplest approach and requires no additional software. systemd handles automatic restarts on failure, log management via journalctl, and service startup on boot.

```
# /etc/systemd/system/renewably.service<br/>[Unit]<br/>Description=Renewably CRM<br/>After=network.target<br/><br/>[Service]<br/>Type=simple<br/>User=renewably<br/>WorkingDirectory=/opt/renewably<br/>Environment=NODE_ENV=production<br/>ExecStart=/home/renewably/.bun/bin/bun .next/standalone/server.js<br/>Restart=always<br/>RestartSec=5<br/>StandardOutput=journal<br/>StandardError=journal<br/><br/>[Install]<br/>WantedBy=multi-user.target
```

```
# Enable and start<br/>sudo systemctl daemon-reload<br/>sudo systemctl enable renewably<br/>sudo systemctl start renewably<br/><br/># View logs<br/>sudo journalctl -u renewably -f
```

Option B: PM2 Process Manager

PM2 provides additional features over systemd including a monitoring dashboard, clustering for multi-core utilisation, and convenient CLI commands for log management. Install PM2 globally and configure it to start on boot using the startup hook.

```
# Install PM2<br/>npm install -g pm2<br/><br/># Start the application<br/>cd /opt/renewably<br/>NODE_ENV=production pm2 start .next/standalone/server.js --name renewably<br/><br/># Generate startup script for auto-restart on reboot<br/>pm2 startup<br/>pm2 save<br/><br/># Useful commands<br/>pm2 logs renewably     # View logs<br/>pm2 monit                # Monitoring dashboard<br/>pm2 restart renewably   # Restart<br/>pm2 stop renewably       # Stop
```

Uptime Monitoring

Set up external uptime monitoring to detect and alert on server downtime. Recommended tools include UptimeRobot (free tier available), Better Uptime, or a custom health check endpoint. The application already exposes a health check at `/api/crm/analytics/website` that returns server status and timing information. Configure monitoring to poll this endpoint every 1-5 minutes and alert the development team on any non-200 response or latency exceeding 5 seconds. For application performance monitoring (APM), consider integrating Datadog or New Relic for request latency tracking and error alerting.